

Encapsulation is your Architecture's defense system

How a single rule: objects never expose their state - naturally produces clean separation of concerns

🕒 6 min read 📁 Article 📅 May 21, 2026



ON THIS PAGE



The full source for this experiment lives at github.com/leprz/encapsulation-workshop-ts.

The repository contains a TypeScript trading simulation built around a single constraint: **objects never expose their state**.

In TypeScript, objects whose internal state is readable from the outside can be expressed in several ways:

- as an interface with properties,
- a type alias of an object literal,
- a class with public fields or getters and setters.

The OOP literature calls these *data structures* or *data objects*; everyday application calls them DTOs, models, schemas.

This codebase intentionally avoids that style in favour of behaviour-oriented objects.

This means:

- No getters.
- No public fields.
- Not even `public readonly`.

Every field on every class is private, which means the only way to interact with an object is through methods it explicitly provides.

Let's explore how this single restriction changes the usual tensions between objects.

A code smell, explained

Feature envy is a code smell named by Martin Fowler:

A method on class A that is more interested in the data of class B than its own.

It keeps calling `b.x`, `b.y`, `b.z` to assemble a decision that really should live on `B`. Changing the properties of `B` suddenly means hunting through their usage in the system - which may be in `A`, `B`, or `C`.

Compare a typical "service-layer" sale:

```
// What this codebase refuses to do
if (customer.wallet.cash.amount >= product.price.amount) { .
```

...with what `Shop.sellProduct` actually looks like:

```
sellProduct(sku: number, buyer: Buyer): void {
  const paidPrice = this.takeProductFromStockBySku(sku).sell
```

```
this.bankAccount.addIncome(paidPrice, `Sale #${sku}`);
}
```

The shop never inspects the buyer's wallet, never reads the product's price. It hands the product to the buyer and says "sell to them." The product asks the buyer "buy this for X." The buyer asks its wallet "withdraw X." Each object only touches state it owns. ►

There are no data-envy objects in the model because no object can see the data of another.

The rule that drives the design

If a class cannot publish its internals, the only way a caller can "do something with" that data is to ask the object to do it.

Look at `Wallet` (`src/banking/Wallet.ts`):

```
export class Wallet {
  constructor(private cash: Money) {}

  withdrawMoney(amount: Money): Money {
    const afterPayment = this.cash.subtract(amount);
    if (afterPayment.isNegative()) {
      throw new WalletNotEnoughCashError('Not enough money');
    }
    this.cash = afterPayment;
    return amount;
  }
}
```

There is no `getCash()`. There cannot be, because the moment such a getter exists, the "is there enough money?" check might migrate outward into callers and the wallet quickly becomes just a data object. The withdrawal rule lives where the cash lives - in the Information Expert.

Who is the Information Expert

GRASP's **Information Expert** pattern says:

Assign a responsibility to the class that has the information needed to fulfil it.

With private-only fields, that becomes the only option.

- `BankAccount` owns the `Transaction[]`, so capital calculation and overdraft commissions live inside it. The `addExpense` method itself decides whether an overdraft commission applies - no external service inspects the balance to figure that out.
- `Product` owns its `sku` and `price`, so `sellTo(buyer)` and `addPriceMargin(margin)` belong to it. The shop never reads `product.price` to compute a markup; it asks the product for a new product with the margin applied.

Every operation in the system is phrased as a verb sent to the object that holds the relevant state.

Collections are where the rule gets interesting

How do you apply this rule to collections of objects? If a caller has a `Product[]` and wants to find one by `sku`, the obvious move is `products.find(p => p.sku === sku)` - but that's blocked because `sku` is private. The Information Expert for that lookup is still `Product`: it is the only place that can read `sku`. So the lookup becomes a static method on the element class:

```
// src/catalog/Product.ts
static findProductBySku(sku: number, products: Product[]): Product {
  return products.find((product) => product.sku === sku);
}
```

A static on `Product` can see the private `sku` of every `Product` instance, so the field never has to be public. Callers ask `Product` to

find the product.

The same pattern applies to aggregations:

```
// src/banking/Transaction.ts
static sumAmounts(transactions: Transaction[]): Money {
  return transactions.reduce((carry, tx) => carry.add(tx.amount), Money.zero)
}
```

And mutations:

```
// src/catalog/Product.ts
static removeFromCollection(product: Product, products: Product[]): Product[] {
  const index = products.indexOf(product);
  if (index !== -1) {
    products.splice(index, 1);
  }
}
```

This has a knock-on effect on application design: the logic on collections is no longer the caller's problem.

How encapsulation naturally produces SOLID

- **Single Responsibility Principle (SRP)** - Each class has one reason to change because each class owns exactly one piece of state. `Wallet` changes when the cash-handling rules change. `BankAccount` changes when the ledger or commission rules change. They don't bleed into each other.
- **Open/Closed Principle (OCP)** - New buyer or supplier types plug in via the `Buyer`, `Reseller`, and `Supplier` interfaces without modifying `Shop`, `Manufacturer`, or `Product`. A `Customer` and a `Shop` are both `Buyer`s, which is why a shop can buy from a manufacturer using the exact same protocol a customer uses to buy from a shop.

- **Liskov Substitution Principle (LSP)** - The interfaces are tiny and behavioural (`buyProduct` , `sellTo` , `receiveStock`). There is no shape to violate.
- **Interface Segregation Principle (ISP)** - `Buyer` has one method. `Supplier` has one method. `Reseller` extends `Buyer` with `receiveStock` . Nothing depends on more than it uses.
- **Dependency Inversion Principle (DIP)** - `Shop` depends on the `Buyer` and `Supplier` abstractions, not on `Customer` or `Manufacturer` concretely. `main.ts` is the only place where concrete classes meet.

A note on the printers

Printing is the one place where domain objects must surface information to the outside world. The design keeps the same rule by inverting the flow: instead of exposing state via getters, each domain object accepts a printer and tells it what to render -

```
wallet.printOn(printer, owner) ,
```

```
bankAccount.printBalanceOn(printer, title) .
```

The printers themselves (`src/printing/*`) are an output concern and are intentionally kept off to the side; they exist so that the encapsulation rule doesn't break at the I/O boundary.

The takeaway

Limiting object state exposure in this codebase makes SOLID emerge naturally because each class ends up with exactly one job: operating on the data it owns. That does not mean everything should always be private. This project intentionally pushes encapsulation further than most production systems would, in order to see what emerges from taking that idea seriously.



Real architecture is not only the product of applying SOLID and GRASP as a checklist.

It is always a balance between forces:

- low vs high coupling,
- direct calls vs streams and events,
- transactions vs eventual consistency.

Good architecture also depends on timing, team structure, ways of working, and business requirements. The hardest part is learning to see the long-term consequences of those decisions early.

Keep reading

[All articles](#)

Turn discovery into understanding

What might be the most valuable skill of this era?

🕒 3 min read 📁 Article

[Read article ↗](#)

Swarming vs Multitasking in Scrum

Why swarming on a single task delivers better results than multitasking across many

🕒 4 min read 📁 Article

[Read article ↗](#)

ABOUT THE AUTHOR



Przemek Łęczycki

Full-Stack Developer • Team Builder

I build software and help teams deliver awesome results.

[View full profile](#) →

© 2026 Created and designed by leprz. All rights reserved.